

Next.js と Django を用いた在庫管理アプリケーションの仕様

k23075 多田 隆人¹

概要：本稿は、Next.js と Django を用いた在庫管理アプリケーションの仕様をまとめたものである。本アプリケーションでは商品の登録、在庫数の確認、仕入れ・卸し操作をブラウザ上で行える。このアプリケーションについてシステム要件、機能設計、画面設計、データベース設計、API 設計、実装環境、環境構築、実装詳細の手順をまとめた。

Specifications for an Inventory Management Application Using Next.js and Django

k23075 TADA RYUTO¹

1. システム要件

本アプリケーションは商品の在庫管理を目的とした Web ベースのシステムである。ユーザは商品の登録、在庫数の確認、仕入れ・卸し操作をブラウザ上で行える。仕入れ・卸し操作ができる他、CSV ファイルをアップロードすると売上数の集計も行える。

ログイン画面、商品一覧画面、商品在庫管理画面、売上一括登録画面がある。ユーザは必ずログイン画面からログインを行う必要があり、ログイン後に商品一覧画面に遷移する。商品一覧画面では登録されている商品の情報を一覧で確認でき、商品情報の追加・編集・削除が行える。商品在庫管理画面では商品ごとに仕入れ数と売上数の管理ができ、在庫の履歴を確認できる。売上一括登録画面では CSV ファイルをアップロードすると年月ごとの売上数集計が表示される。

2. 機能設計

機能設計では、アプリケーションが提供する機能を詳細に定義する。機能ごとに必要な要件を示す。

ログイン機能があり、ユーザはアプリケーションにログインして利用を開始する。データベース上でユーザ情報を管理し、ログイン時に認証を行う。15 分で自動的にログアウトを行い、セキュリティを確保する。今回は事前に shell

上でユーザ登録を行う。

ログアウト機能があり、ユーザはアプリケーションからログアウトできる。ログアウト後は再度ログイン機能を使用してログインする必要がある。

商品一覧機能があり、登録されている商品の情報を一覧で確認できる。商品 ID、商品名、単価、説明、商品在庫管理へのリンク、編集ボタンを表示する。

商品追加機能があり、商品情報の追加ができる。商品一覧機能の右上部に商品追加ボタンがあり、クリックすると商品追加エリアが表示する。商品名、単価、説明を入力し、登録ボタンをクリックすると商品情報が登録する。商品名と単価は必須項目であり、未入力の場合は登録できない。単価は 1 円～99999999 円の範囲で入力する必要がある。入力が正しくない場合はエラーメッセージが表示する。商品 ID は自動的に生成する。

商品編集機能があり、商品情報の編集ができる。商品名、単価、説明を入力し、更新ボタンをクリックすると商品情報を更新する。編集後に更新ボタンをクリックすると商品情報を更新する。商品追加機能同様に入力が正しくない場合はエラーメッセージを表示する。

商品削除機能があり、商品情報の削除ができる。商品削除ボタンをクリックすると商品情報を削除する。

在庫管理機能があり、商品ごとに仕入れ数と売上数の管理できる。数量を入力し、商品仕入れボタンもしくは商品卸しボタンをクリックすると、商品在庫数を更新する。1

¹ 愛知工業大学 情報科学部
Department of Information Science, Aichi Institute of Technology

から 99999999 までの範囲で入力する必要がある。入力が正しくない場合はエラーメッセージを表示する。

在庫履歴機能があり、商品ごとの在庫履歴を確認できる。在庫履歴は処理種別（仕入れ or 卸し）、処理日時、単価、数量、価格、在庫数を表示する。在庫履歴は最新のものから順に表示する。

売上一括登録機能があり、CSV ファイルをアップロードすると年月ごとの売上数集計を登録できる。同期処理、非同期処理の両方行える。CSV ファイルは product, date, quantity の 3 つの項目が必要である。

売上数集計機能があり、年月ごとの売上数を集計して表示する。集計は売上一括登録機能で登録されたデータと在庫管理機能で登録されたデータを元に行う。集計結果は年月と売上数を表示する。

3. 画面設計

画面設計では、アプリケーションの各画面のレイアウトと機能を定義する。画面ごとに備える機能とその配置を示す。

ログイン画面を除く各画面はヘッダー・フッター・サイドバーを持ち、ログアウトと画面遷移を行う。ヘッダーにはアプリケーション名とログアウトボタンが配置される。ログアウトボタンをクリックするとログアウトが行われる。フッターにはコピーライト情報が表示される。サイドバーには商品一覧画面と売上一括登録画面へのリンクが配置される。

ログイン画面では、認証機能を備える。画面上にはユーザ名とパスワードの入力欄があり、ログインボタンが配置される。ユーザ名とパスワードを入力後にログインボタンをクリックすると認証が行われ、成功した場合は商品一覧画面に遷移する。ユーザ名とパスワードは必須項目であり、未入力の場合はエラーメッセージが表示される。パスワードは 8 文字以上である必要があり、8 文字未満の場合はエラーメッセージが表示される。また、ログイン失敗時にもエラーメッセージが表示される。

商品一覧画面では、商品一覧機能・商品追加機能・商品編集機能・商品削除機能を備える。画面上には表で商品一覧が表示される。表左上部に商品追加ボタンがあり、クリックすると商品追加エリアが表示される。商品情報の編集は商品一覧画面の各商品情報の右側の編集ボタンをクリックすると行える。このとき削除ボタンも表示され、クリックすると商品情報が削除される。商品追加・削除時に入力が正しくない場合はテキストエリア下部にエラーメッセージが表示される。正常に登録・更新・削除が行われた場合は商品一覧が更新され、画面左下部にメッセージが表示される。表示数に制限はなく、全ての商品を表示する。

商品在庫管理画面では、在庫管理機能と在庫履歴機能を備える。画面上部には商品名と数量の入力欄があり、商品

表 1 商品テーブルのスキーマ定義

カラム名	データ型	制約
id	bigint	PK, AUTOINCREMENT
name	varchar(100)	NOT NULL
price	int	NOT NULL
description	longtext	

表 2 仕入れテーブルのスキーマ定義

カラム名	データ型	制約
id	bigint	PK, AUTOINCREMENT
product_id	bigint	FK (product.id)
quantity	int	NOT NULL
purchase_date	datetime	NOT NULL

仕入れボタンと商品卸しボタンが配置される。数量が正しくない場合は入力欄下部にエラーメッセージが表示される。正常に仕入れ・卸しが行われた場合は在庫数が更新され、画面左下部にメッセージが表示される。画面下部には在庫履歴が表形式で表示される。在庫履歴は最新のものから順に表示される。表示数に制限はなく、全ての履歴を表示する。

売上一括登録画面では、売上一括登録機能と売上数集計機能を備える。画面上には年月ごとの売上数集計の一覧とファイル取込みエリアが表示される。ファイル取込みエリアには同期でファイル取込みと非同期でファイル取込みの 2 つがある。それぞれファイル選択エリアと登録ボタンが配置される。同期で登録した場合は、登録完了後に売上数集計が表示される。非同期で登録した場合は、瞬時に売上数集計の一覧が更新され、登録完了後に再度売上数集計が更新される。

4. データベース設計

本アプリケーションではデータベースで商品情報、仕入れ・売上履歴、売上ファイルの管理を行う。テーブルごとに説明する。

商品テーブルでは商品名、単価、説明といった商品情報を管理する。商品テーブルのスキーマ定義を表 1 に示す。id は主キーとして自動で連番が振られる。name と price は必須項目である。

仕入れテーブルでは商品ごとの仕入れ数を管理する。仕入れテーブルのスキーマ定義を表 2 に示す。id は主キーとして自動で連番が振られる。product_id は商品テーブルの id を参照する外部キーである。また、仕入れ数は必須項目である。

売上ファイルテーブルでは売上ファイルの情報を管理する。売上ファイルテーブルのスキーマ定義を表 3 に示す。id は主キーとして自動で連番が振られる。status は売上ファイルの状態を表しており、実際の値との対応を表 4

表3 売上ファイルテーブルのスキーマ定義

カラム名	データ型	制約
id	bigint	PK, AUTOINCREMENT
file_name	varchar(100)	NOT NULL
status	int	NOT NULL

表4 売上ファイルの状態

値	状態
0	同期
1	非同期 (未処理)
2	非同期 (処理済)

に示す。

5. API 設計

API 設計では、アプリケーションが提供する API のエンドポイントとその機能を定義する。

5.1 ログイン API

ログイン API は JWT(JSON Web Token) [1] を用いてユーザ認証を行うためのエンドポイントである。エンドポイントは `/api/inventory/login` であり、POST メソッドのみを受け取る。リクエストボディにはユーザ名とパスワードを含む JSON オブジェクトを送信する。レスポンスとしては、認証に成功した場合は JWT トークンを Cookie に保存し、HTTP ステータスコード 200 OK を返す。また、Cookie に JWT トークンを保存し、次回以降のリクエストで認証を行う。認証に失敗した場合はエラーメッセージを含む JSON オブジェクトを返す。このとき、HTTP ステータスコードは 401 Unauthorized を返す。

5.2 ログアウト API

ログアウト API はクライアント側の Cookie から JWT トークンを削除してログアウトを行うためのエンドポイントである。エンドポイントは `/api/inventory/logout` であり、POST メソッドのみを受け取る。リクエストボディは不要であり、HTTP ステータスコード 200 OK のみを返す。

5.3 リトライ API

リトライ API は非同期処理の再実行を行うためのエンドポイントである。エンドポイントは `/api/inventory/retry` であり、POST メソッドのみを受け取る。HTTP ヘッダー `HTTP_REFRESH_TOKEN` にはリフレッシュトークンを含める必要がある。認証に成功した場合は、HTTP ステータスコード 200 OK を返し、新しい JWT トークンを Cookie に保存する。認証に失敗した場合はエラーメッセージを含む JSON オブジェクトを返し、HTTP ステータスコード 401 Unauthorized を返す。

5.4 商品一覧 API

商品一覧 API は登録されている商品の情報を取得するためのエンドポイントである。エンドポイントは `/api/inventory/products/product_id?` であり、GET, POST, PUT, DELETE メソッドを受け取る。

GET メソッドでは全ての商品情報を取得し、JSON 形式でレスポンスを返す。また、パスパラメータとして商品 ID を指定すると、特定の商品情報を取得可能である。商品数に制限はなく、全ての商品を返す。

POST メソッドでは新しい商品情報を登録するためのエンドポイントである。リクエストボディには商品名、単価、説明を含む JSON オブジェクトを送信する。登録に成功した場合は、HTTP ステータスコード 201 Created を返し、新しい商品情報の JSON オブジェクトをレスポンスとして返す。

PUT メソッドでは既存の商品情報を更新するためのエンドポイントである。パスパラメータとして更新する商品 ID を指定する必要がある。リクエストボディには更新する商品情報を含む JSON オブジェクトを送信する。更新に成功した場合は、HTTP ステータスコード 200 OK を返し、更新後の商品情報の JSON オブジェクトをレスポンスとして返す。

DELETE メソッドでは特定の商品情報を削除するためのエンドポイントである。パスパラメータとして削除する商品 ID を指定する必要がある。削除に成功した場合は HTTP ステータスコード 200 OK を返す。

5.5 仕入れ・売上取得 API

仕入れ・売上取得 API は商品ごとの仕入れ数と売上数の管理を行うためのエンドポイントである。エンドポイントは `/api/inventory/inventories/{product_id}` であり、GET メソッドのみを受け取る。商品 ID をパスパラメータとして指定する必要がある。商品 ID が存在しない場合は HTTP ステータスコード 404 Not Found を返す。リクエストに成功した場合は、商品ごとの仕入れ情報と売上情報を含む JSON オブジェクトをレスポンスとして返す。

5.6 仕入れ登録 API

仕入れ登録 API は商品ごとの仕入れ数を登録するためのエンドポイントである。エンドポイントは `/api/inventory/purchase` であり、POST メソッドのみを受け取る。リクエストボディには商品 ID、仕入れ数、仕入れ日時を含む JSON オブジェクトを送信する。登録に成功した場合は、HTTP ステータスコード 201 Created を返し、登録された仕入れ情報の JSON オブジェクトをレスポンスとして返す。

5.7 売上登録 API

売上登録 API は商品ごとの売上数を登録するためのエンドポイントである。エンドポイントは `/api/inventory/sales` であり、POST メソッドのみを受け取る。リクエストボディには商品 ID、売上数、売上日時を含む JSON オブジェクトを送信する。登録に成功した場合は、HTTP ステータスコード 201 Created を返し、登録された売上情報の JSON オブジェクトをレスポンスとして返す。

5.8 売上ファイル登録 API

売上ファイル登録 API は CSV ファイルをアップロードして売上数の集計を行うためのエンドポイントである。同期処理と非同期処理の 2 つを提供する。同期処理のエンドポイントは `/api/inventory/sales/sync` であり、POST メソッドのみを受け取る。非同期処理のエンドポイントは `/api/inventory/sales/async` であり、POST メソッドのみを受け取る。リクエストボディには CSV ファイルを含むマルチパートフォームデータを送信する。同期処理では、CSV ファイルを解析して売上数を集計し、HTTP ステータスコード 201 Created を返す。非同期処理では、CSV ファイルを保存し、即座に HTTP ステータスコード 201 Created を返す。その後、バッチ処理で非同期に売上数の集計を行い、処理が完了したら売上ファイルテーブルのステータスを更新する。

5.9 売上数集計 API

売上数集計 API は年月ごとの売上数を集計して表示するためのエンドポイントである。エンドポイントは `/api/inventory/sales/summary` であり、GET メソッドのみを受け取る。リクエストボディは不要であり、レスポンスとしては年月とその売上数を含む JSON オブジェクトを返す。

6. 実装

6.1 実装環境

本アプリケーションは Next.js [2] と Django [3] を用いて実装する。Next.js は React [4] ベースのフレームワークであり、サーバサイドレンダリングや静的サイト生成をサポートしている。Django は Python ベースの Web フレームワークであり、迅速な開発とセキュリティを提供する。以下に実装環境の詳細を示す。

- Mac OS: 14.6.1
- Docker: 28.0.1
- Node.js: v22.16.0
- Next.js: 15.1.1
- React: 19.0.0
- Emotion: 11.14.0

- Material UI: 7.1.0
- TypeScript: 5.2.2
- Django: 5.2.1
- Python: 3.13.3
- MySQL: 8.4.5

6.2 環境構築

本アプリケーションは Docker Compose [5] を用いてコンテナ化する。Docker Compose を使用すると、複数のコンテナを一括で管理できるため、開発環境の構築が容易になる。frontend, backend, db の 3 つのサービスを定義する。

6.2.1 データベース

db サービスは MySQL を実行するコンテナであり、ポート 53306 を公開する。healthcheck を使用して MySQL の起動を確認する。mysqladmin ping コマンドを使用して MySQL が起動しているかを確認し、5 秒ごとに最大 5 回まで確認を行う。

環境変数を使用して MySQL の初期設定を行う。環境変数を用いると、コンテナ起動時に設定を簡単に変更できる [6]。設定する環境変数は以下の通りである。

- MYSQL_ROOT_PASSWORD: root パスワード
- MYSQL_DATABASE: 初期データベース名
- TZ: タイムゾーン

6.2.2 バックエンド

backend サービスは Django を実行するコンテナであり、ポート 8000 を公開する。docker image は python:3.13-bullseye [7] を使用する。volumes を使用してデータの永続化とホストマシンにおけるホットリロードを実現する。また、depends_on を使用して db サービスが起動してから backend サービスが起動するようにする。ENTRYPOINT には python manage.py runserver 0.0.0.0:8000 --settings config.settings.development を指定する。

django-admin コマンドを使用してプロジェクトを作成する。django-admin startproject config . コマンドを実行しプロジェクトを作成すると、manage.py ファイルと config ディレクトリが生成される。django-admin startapp inventory コマンドを実行しアプリケーションを作成すると、inventory ディレクトリが生成される。settings.py ファイルに以下の設定を追加する。

- INSTALLED_APPS: inventory アプリケーションを追加する。
- DATABASES: db サービスの MySQL データベースを使用するように設定する。
- LANGUAGE_CODE: 'ja-jp' を指定し、日本語を使用する。
- LOGGING: ログ出力の設定を追加する。

6.2.3 フロントエンド

frontend サービスは Next.js を実行するコンテナであり、

表 5 型ごとのフィールド定義

データ型	フィールド名
int	IntegerField
bigint	BigIntegerField
varchar	CharField
longtext	TextField
datetime	DateTimeField
FK	ForeignKey

ポート 3000 を公開する。docker image は `node:22-slim` [8] を使用する。volumes を使用してデータの永続化とホストマシンにおけるホットリロードを実現する。ENTRYPOINT には `yarn dev` を指定する。

`npx create-next-app@15.1.1` [9] コマンドを実行しプロジェクトを作成する。また、今回は App Router [10] と Material UI [11] を使用する。

App Router [10] は Next.js の新しいルーティングシステムであり、ページベースのルーティングを提供する。コンポーネントごとにクライアントコンポーネントとサーバコンポーネント [12] を選択できる。これによりページごとに異なるデータフェッチング戦略を選択できるため、柔軟なアプリケーションの構築が可能となる。

Material UI [11] は Google の Material Design に基づいた React コンポーネントライブラリである。スタイリングとコンポーネントの再利用性を向上させるために使用する。

Next.js の Rewrites [13] を使用してリクエストをバックエンドの Django サーバーに転送する。 `/api/:path*` のリクエストを `http://host.docker.internal:8000/api/:path*/` に転送するように設定する。これにより CORS エラーを回避してバックエンドの API にアクセスできる。

7. 実装詳細

7.1 モデルの実装

データベースのテーブルを Django のモデルとして実装する方法を示す。モデルは `models.py` ファイルに `Model` クラスを継承して実装する。

型ごとにフィールドを用いて定義する。 `null=True` `blank=True` を指定すると NULL 値を許容し、任意項目にできる。また `verbose_name` を指定するとフィールドの表示名を設定できる。型ごとのフィールド定義を表 5 に示す。

`IntegerChoices` を使用して定数を定義し、選択肢を設定できる。変数名は大文字で定義し、実際の値と説明を設定する。`IntegerField` に対して `choices` に指定すると選択肢を設定できる。

7.2 API の実装

Django REST Framework を使用して API エンドポイントを実装する方法を示す。API は `views.py` ファイルに実装する。

7.2.1 ログイン・ログアウト機能

ユーザ認証には `django-rest-framework-simplejwt` [14] を用いて JWT を使用する。`settings.py` の `REST_FRAMEWORK.DEFAULT_AUTHENTICATION_CLASSES` に `AccessJWTAuthentication` と `JWTAuthentication` を追加し、JWT 認証を有効にする。`REST_FRAMEWORK.DEFAULT_PERMISSION_CLASSES` には `IsAuthenticated` を指定し、認証済みのユーザのみが API にアクセスできるようにする。`SIMPLE_JWT` の設定を行い、トークンの有効期限を 15 分に設定する。`COOKIE_TIME` を設定し、Cookie の有効期限を 15 分に設定する。また、`TIME_ZONE` を `Asia/Tokyo` に設定し、タイムゾーンを日本に合わせる。これにより、JWT トークンの検証を行う際にタイムゾーンの問題を回避できる。

ログイン時には `TokenObtainPairSerializer` によるシリアライザを定義し、ユーザ名とパスワードを検証する。`serializer.is_valid` を用いて検証を行う。検証に成功した場合は JWT トークンを生成し、`response.set_cookie` を用いて Cookie に保存する。検証に失敗した場合は例外処理を行い、401 Unauthorized エラーを返す。

ログアウト時には Cookie から JWT トークンを削除する。`response.delete_cookie` を使用して Cookie からトークンを削除する。

7.2.2 商品一覧・追加・編集・削除機能

商品を一覧取得・追加・編集・削除する API を実装する。各 HTTP メソッドごとにクラスメソッドを定義する。Product テーブルは `Product` モデルを使用して操作する。`ModelSerializer` を継承した `ProductSerializer` を使用して商品情報をシリアライズする。

GET メソッドでは全ての商品情報を取得するためのエンドポイントを定義する。`Product.objects.all()` を使用して全ての商品情報を取得し、JSON 形式でレスポンスを返す。また、商品 ID をパスパラメータとして指定した場合は `Product.objects.get()` を使用して特定の商品情報を取得する。

POST メソッドでは新しい商品情報を登録するためのエンドポイントを定義する。リクエストボディには商品名、単価、説明を含む JSON オブジェクトを送信する。`ProductSerializer` を使用してリクエストデータをシリアライズし、`serializer.is_valid()` を使用して検証を行う。検証に成功した場合は `serializer.save()` を使用して商品情報を登録し、HTTP ステータスコード 201 Created を返す。

PUT メソッドでは既存の商品情報を更新するためのエン

ドポイントを定義する。パスパラメータとして削除する商品 ID を指定し、リクエストボディには更新する商品情報を含む JSON オブジェクトを送信する。 `Product.objects.get()` を使用して更新する商品情報を取得し、 `ProductSerializer` を使用してリクエストデータをシリアル化し検証を行う。検証に成功した場合は `serializer.save()` を使用して商品情報を更新し、HTTP ステータスコード 200 OK を返す。

DELETE メソッドでは特定の商品情報を削除するためのエンドポイントを定義する。パスパラメータとして削除する商品 ID を指定する。 `Product.objects.get()` を使用して削除する商品情報を取得し、 `product.delete()` を使用して商品情報を削除する。

7.2.3 在庫管理機能

商品仕入れ・卸し API を実装する。それぞれ `APIView` クラスを継承した `PurchaseView` と `SaleView` を定義する。 `ModelSerializer` を継承した `PurchaseSerializer` と `SaleSerializer` を使用して仕入れ・売上情報をシリアル化し検証を行う。仕入れ時は検証に成功した場合は `PurchaseSerializer.save()` を使用して仕入れ情報を登録し、HTTP ステータスコード 201 Created を返す。卸し時は検証に成功した場合は在庫数を超えないかを確認したのちに卸し情報を登録し、HTTP ステータスコード 201 Created を返す。在庫数を超えた場合は `BusinessException` を用いてエラーメッセージを返す。

7.2.4 在庫履歴機能

在庫履歴 API を実装する。 `ListAPIView` クラスを継承した `SalesList` を定義する。 `Sales.objects.annotate()` を使用し月毎に集計した在庫履歴を取得し、 `Sum('quantity')` を使用して数量を集計する。 `SalesSerializer` を使用して在庫履歴をシリアル化し、JSON 形式でレスポンスを返す。

7.2.5 売上一括登録機能

売上一括登録 API を同期処理として実装する。 `Serializer` を継承した `FileSerializer` を使用して CSV ファイルをシリアル化し検証を行う。検証に成功した場合は `open()` 関数を使用して CSV ファイルを保存する。 `SalesFile` モデルを使用して同期ファイルとしてファイル情報を登録する。 `pandas` ライブラリを使用して CSV データを `DataFrame` に変換し、売上数を集計する。集計結果は `Sales` モデルを使用して登録し、HTTP ステータスコード 201 Created を返す。

売上一括登録 API を非同期処理として実装する。同期処理と同様に `open()` 関数を使用して CSV ファイルを保存する。検証に成功した場合は `open()` 関数を使用して CSV ファイルを保存する。 `SalesFile` モデルを使用して非同期ファイルとしてファイル情報を登録する。この時点で HTTP ステータスコード 201 Created を返す。その後、バッチ処理を用いて非同期に売上数の集計を行う。 `BaseCommand`

を継承した `Command` クラスを定義し、 `handle()` メソッドで非同期処理を実装する。 `SalesFile.objects.filter()` を使用して未処理の非同期ファイルを取得し、 `pandas` ライブラリを使用して CSV データを `DataFrame` に変換する。集計結果は `Sales` モデルを使用して登録し、 `SalesFile` モデルのステータスを更新する。

7.3 フロントエンドの実装

Next.js を使用してフロントエンドを実装する方法を示す。

7.3.1 ディレクトリ構成

Next.js の App Router を使用したディレクトリ構成を示す。 `app` ディレクトリ以下にページやコンポーネントを配置する。ファイルベースのルーティングを使用し、ディレクトリ名がそのまま URL パスとなる。 `layout.tsx` ファイルではそのファイルの階層以下のページのレイアウトを定義できる [15]。 `page.tsx` ファイルではそれぞれのページのコンテンツを定義できる。またフォルダ名に `[id]` のように指定すると動的ルーティングが可能となる。

7.3.2 フェッチャーの実装

API との通信を行うフェッチャーを実装する。フェッチャーは `plugins/axios.ts` ファイルに実装する。 `axios` ライブラリ [16] を使用して API との通信を行う。

`axios.create()` を使用してインスタンスを作成する。 `headers` に `Content-Type: application/json` を指定し、JSON 形式でリクエストを送信する。

レスポンスインターセプターでは成功時と失敗時のハンドリングを行う。成功時にはレスポンスデータをそのまま返す。401 Unauthorized エラーが発生した場合はリフレッシュトークンを使用して再認証を行う。このとき、ログイン処理の場合は再認証は行わずエラーを返す。再認証失敗時にはログイン画面にリダイレクトする。

7.3.3 ログイン画面

ログイン画面は `app/login/page.tsx` に実装する。フォーム要素は Material UI の `TextField` コンポーネントと `Button` コンポーネントを使用して実装する。入力情報は `react-hook-form` [17] の `useForm` を使用して管理する。ユーザ名やパスワードの入力欄が未入力の場合や適切でない場合は `register` を用いて検証を行いエラーメッセージを表示する。ログインボタンをクリックすると `handleSubmit` を使用してフォームの送信を行う。送信時にはフェッチャーを使用してログイン API にリクエストを送信する。

7.3.4 レイアウト

レイアウトは `app/inventory/layout.tsx` に実装する。レイアウトではヘッダー・フッター・サイドバーを含む共通のレイアウトを定義する。サイドバーのリンクは Material UI の `List` コンポーネントを使用して実装する。

7.3.5 商品一覧画面

商品一覧画面は `app/inventory/products/page.tsx` に実装する。商品情報を取得するために `useEffect` フックを使用してフェッチャーを呼び出す。取得した商品情報は Material UI の `TableContainer` コンポーネントを使用して表形式で表示する。商品追加フォームは Material UI の `TextField` コンポーネントと `Button` コンポーネントを使用して実装する。入力情報は `react-hook-form` の `useForm` を使用して管理する。入力情報が適切でない場合は `register` を用いて検証を行いエラーメッセージを表示する。登録ボタンをクリックすると `handleSubmit` を使用してフォームの送信を行う。送信時にはフェッチャーを使用して商品一覧 API にリクエストを送信する。

`useState` フックを用いて編集時の商品 ID を管理する。初期値は `null` とする。編集ボタンをクリックすると編集時の商品 ID を設定し、ID が一致する商品情報をフォームに表示する。登録・更新・キャンセル時には再び `null` を設定する。また、処理の状態 (`add`, `edit`, `delete`) を `useState` フックで管理する。完了ボタンをクリックすると処理の状態に応じて商品情報を登録・更新・削除する。処理の状態に応じてフェッチャーを使用して商品一覧 API にリクエストを送信する。

7.3.6 在庫管理画面

在庫管理画面は `app/inventory/products/[id]/page.tsx` に実装する。`useParams` フックを使用して商品 ID を取得する。`useEffect` フックを使用して初期レンダリング時に在庫情報を取得する。在庫情報は Material UI の `TableContainer` コンポーネントを使用して表形式で表示する。

在庫処理フォームは Material UI の `TextField` コンポーネントと `Button` コンポーネントを使用して実装する。入力情報は `react-hook-form` の `useForm` を使用して管理する。入力情報が適切でない場合は `register` を用いて検証を行いエラーメッセージを表示する。仕入れボタンもしくは卸しボタンをクリックすると `handleSubmit` を使用してフォームの送信を行う。送信時にはフェッチャーを使用して仕入れ・売上取得 API にリクエストを送信する。

7.3.7 売上一括登録画面

売上一括登録画面は `app/inventory/import_sales/page.tsx` に実装する。`useEffect` フックを使用して初期レンダリング時に売上情報を取得する。売上情報は Material UI の `TableContainer` コンポーネントを使用して表形式で表示する。

Material UI の `MuiFileInput` コンポーネントを使用してファイル選択エリアを実装する。登録ボタンをクリックすると `handleSubmit` を使用してフォームの送信を行う。送信時にはフェッチャーを使用して売上ファイル登録 API にリクエストを送信する。

8. まとめ

本稿では、Next.js と Django を用いた在庫管理アプリケーションの仕様をまとめた。本アプリケーションでは商品の登録、在庫数の確認、仕入れ・卸し操作をブラウザ上で行える。このアプリケーションについてシステム要件、機能詳細、データベース設計、実装環境、環境設定、動作検証の結果をまとめた。今後の課題としては、ユーザ管理機能の追加が挙げられる。現在は事前に shell 上でユーザ登録を行う必要があるが、Web 上でユーザ登録を行えるようにする必要がある。

参考文献

- [1] auth0: JWT, <https://jwt.io/> (2025).
- [2] Vercel: Next.js, <https://nextjs.org/> (2025).
- [3] Django Software Foundation: Django, <https://www.djangoproject.com/> (2025).
- [4] Meta Platforms, I.: React, <https://react.dev/> (2025).
- [5] Docker: Docker Compose, <https://docs.docker.com/compose/> (2025).
- [6] Docker: mysql - Official Image, https://hub.docker.com/_/mysql/ (2025).
- [7] Docker: python - Official Image, https://hub.docker.com/_/python/ (2025).
- [8] Docker: node - Official Image, https://hub.docker.com/_/node/ (2025).
- [9] Vercel: CLI: create-next-app, <https://nextjs.org/docs/app/api-reference/cli/create-next-app> (2025).
- [10] Vercel: Introduction: App Router, <https://nextjs.org/docs/app> (2025).
- [11] MUI: MUI, <https://mui.com/> (2025).
- [12] Meta Platforms, I.: Server Components, <https://react.dev/reference/rsc/server-components> (2025).
- [13] Vercel: next.config.js: rewrites, <https://nextjs.org/docs/app/api-reference/config/next-config-js/rewrites> (2025).
- [14] jazzband: Simple JWT, <https://django-rest-framework-simplejwt.readthedocs.io/en/latest/> (2025).
- [15] Vercel: Getting Started: Layouts and Pages, <https://nextjs.org/docs/app/getting-started/layouts-and-pages> (2025).
- [16] Axios: Axios, <https://axios-http.com/ja/> (2025).
- [17] BEEKAI: React Hook Form, <https://react-hook-form.com/> (2025).